

Bayesian LSTM Benchmark for Uncertainty Quantification in Time-Series Process

Billy Hernawan^{1*}

^{1*}Civil & Environmental Engineering, Texas A&M University.

Corresponding author(s). E-mail(s): hernawa2@tamu.edu;

Abstract

This study explores the use of a Bayesian Long Short-Term Memory approach based on the Multiple-Try Metropolis sampler. The proposed method is then compared to autoregression model and the Gaussian Process regression model to compare and contrast the similarity. Furthermore, this study adopted dataset that shows clear distinguishable trend, seasonality and noise in order to full capture the model capability and limitation in working with near "ideal" dataset.

Keywords: LSTM, Gaussian Process Regression, Autoregression, Seasonality

1 Introduction

The treatment of time series dataset that contains either observable or non-observable pattern is quite different from other type of dataset in which Y_t depends on previous values up to Y_{t-s} where s can be defined as lag. Time series analysis as a stochastic process boils down to inferring the data-generating process and predicting the unobserved realization of the data in future time [1]. Nonetheless, predicting independent variable is quite challenging due to that most observable data is merely a limited realization (i.e., sample) from the joint distribution $P_{Y,X}$ in the multivariate case, which is typically unknown.

Since the introduction of the Long Short-Term Memory (LSTM) network architecture [2], Recently, LSTM has seen popularity in time-series forecasting [3]. However, from the probabilistic point of view, a trained data-driven model yield the expected value (i.e., point estimate) of the dependent variables given the independent variables. This can be interpreted as "extrapolation" of the trend within the observed time series instead of forecasting, where the uncertainty of the process must be incorporated.

Recently, there has been significant efforts to shift towards uncertainty quantification in neural network. Jospin et al. [4] highlighted differences and similarities between point estimate neural network and Bayesian neural network. Furthermore, the authors investigated the main roadblocks of *training* a neural network using Bayesian approach. Chandra & He [5] recently investigated the use of parallel computing system to train a Bayesian neural network model for stock price forecasting.

This study proposes the use of Multiple-try Metropolis (MTM) sampler [6], a variant of the M-H sampler, to approximate the full posterior of the LSTM model architecture. MTM leverages multiple independent and identically distributed samples from the proposal distribution and evaluates each candidate proposal by computing their associated weights. Furthermore, a univariate autoregressive model and nonparametric Bayesian model were also applied on the dataset to compare the capability of each approach.

2 Methodology

2.1 Long-Short Term Memory

LSTM is a type of artificial neural network specifically designed to process and analyze sequential data, such as time series, speech, or text. It is a variation of a Recurrent Neural Network (RNN), designed to address the issue of both vanishing and exploding gradient (due to sequential multiplication of gradient during back-propagation process) and RNN's inability to effectively learn long-term dependencies. Each LSTM cell consists of the Forget Gate, Input Gate and Output Gate along with both old and updated Long-Term Memory coefficient. The model can be summarized as

$$\begin{aligned}
\text{Forget gate: } f_t &= \sigma(W_{if} \cdot x_t + b_{if} + W_{hf} \cdot h_{t-1} + b_{hf}) \\
\text{Input gate: } i_t &= \sigma(W_{ii} \cdot x_t + b_{ii} + W_{hi} \cdot h_{t-1} + b_{hi}) \\
\text{Candidate values: } g_t &= \tanh(W_{ig} \cdot x_t + b_{ig} + W_{hg} \cdot h_{t-1} + b_{hg}) \\
\text{Output gate: } o_t &= \sigma(W_{io} \cdot x_t + b_{io} + W_{ho} \cdot h_{t-1} + b_{ho}) \\
\text{Update current cell state: } C_t &= f_t \odot C_{t-1} + i_t \odot g_t \\
\text{Hidden state: } h_t &= o_t \odot \tanh(C_t)
\end{aligned}$$

The parameters of interest in this study are the associated weights and biases at each gate with respect to both input and hidden state.

2.2 Multiple-Try Metropolis (MTM)

The Multiple-Try Metropolis (MTM) algorithm is an extension of the Metropolis-Hastings algorithm used in Markov Chain Monte Carlo (MCMC) sampling, designed to improve efficiency by proposing multiple candidate states at each step instead of a single one. In MTM, a set of candidates is drawn from a proposal distribution, and one candidate is selected probabilistically based on weights derived from the target distribution. This increases the chance of moving to states with higher probabilities,

improving convergence and sampling efficiency. The algorithm includes a correction term to ensure detailed balance and maintain the validity of the Markov chain. MTM is particularly useful in high-dimensional or complex sampling problems where vanilla Metropolis-Hastings often struggles with low acceptance rates. The algorithm is highlighted below.

Algorithm 1 Multiple-Try Metropolis (MTM) Algorithm

- 1: Initialize the Markov chain at state x_0
- 2: **for** $t = 0, 1, 2, \dots$ **do**
- 3: **Propose multiple candidates:** Generate m candidate states $\{x'_1, x'_2, \dots, x'_m\}$ from a proposal distribution $q(x|x_{t-1})$
- 4: **Compute weights:** Evaluate the weights for each candidate:

$$w'(x'_i) = \pi(x'_i)L(g(x'_i)|x'_i), \quad \text{for } i = 1, \dots, m$$

- 5: **Select a candidate:** Choose one candidate x'_j from $\{x'_1, x'_2, \dots, x'_m\}$ with probability proportional to the weight $w'(x'_j)$
- 6: **Generate reverse candidates:** Generate m reverse states $\{x''_1, x''_2, \dots, x''_m\}$ from $q(x|x'_j)$
- 7: **Compute reverse weights:** Evaluate the weights for reverse candidates:

$$w''(x''_i) = \pi(x''_i)L(g(x''_i)|x''_i), \quad \text{for } i = 1, \dots, m$$

- 8: **Calculate acceptance probability:**

$$\alpha = \min \left(1, \frac{\sum_{i=1}^m w'(x'_i)}{\sum_{i=1}^m w''(x''_i)} \right)$$

- 9: **Accept or reject:**
 - 10: **if** $u \sim \text{Uniform}(0, 1) \leq \alpha$ **then**
 - 11: Set $x_{t+1} = x'_j$
 - 12: **else**
 - 13: Set $x_{t+1} = x_t$
 - 14: **end if**
 - 15: **end for**
-

Discussion on the selection of the prior and likelihood depends on the experimental design and the case study. They are discussed in the following section of the dataset.

2.3 Autoregression (AR)

Autoregressive model seek to express the current value as a linear combination of its past values, parameterized by a set of θ . In an AR model of order p , denoted as $\text{AR}(p)$, the value at a time t , $y(t)$ is modeled as

$$y_t = \sum_{i=1}^p \phi_i y_{t-i} + \epsilon_t$$

where ϕ_i are the linear coefficient that captures temporal dependencies, and ϵ_t is a random error term. p is also known as the "lag" of the AR model. Each lag represents how many time steps back the model looks to predict the current value. The model captures how previous observations influence the current one, making it particularly useful for forecasting stationary time series data, where trends and seasonality are stable over time.

2.4 Gaussian Process Regression (GPR)

Gaussian Process Regression is a Bayesian non-parametric model that defines a distribution over functions rather than parameters. It assumes that any finite collection of function values follows a multivariate normal (Gaussian) distribution. Formally, a Gaussian process is defined as

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

where $m(x) = \mathbb{E}[f(x)]$ is the mean function and $k(x, x') = \text{Cov}(f(x), f(x'))$ is the covariance (i.e., kernel) function that encodes the similarity between x and x' .

2.5 Dataset and Experiment Design

The dataset selected for this study is a uni-variate time-series that shows clear monotonically increasing trend and distinct seasonal pattern. The dataset is shown in Fig 1 along with its Seasonal-Trend decomposition using LOESS (STL).

To determine the distribution of randomness that is intended to be captured, the residual of the time-series is studied and the underlying distribution is given in Fig 2. Furthermore, the correlation plot for a lag value of one is presented in Fig 3. It is clear that there is a strong correlation with a lag value of one. Autocorrelation with lag reveals the temporal dependencies in the data, helping identify patterns such as periodicity or smoothness. These insights guide the selection kernels. Fig 4 shows the autocorrelation applied only on the seasonality and residual component of the timeseries data. From here, a cyclic behavior of 12 period step was determined. Consequently, the kernel design selected is the product of a Radial Basis Function (i.e., to model the Gaussian noise) and a periodic kernel (i.e., exponential sine squared).

Probabilistically, the LSTM model is a function that maps n discrete values (i.e., lookback value) at time t to a future value of $t + g$ where g is some time step away. That is, the forward model (LSTM) yields the expected value at time $t + g$. From this perspective, any parameterized Bayesian predictive model results in the distribution

Fig. 1 Airflight Passenger Time-Series Dataset

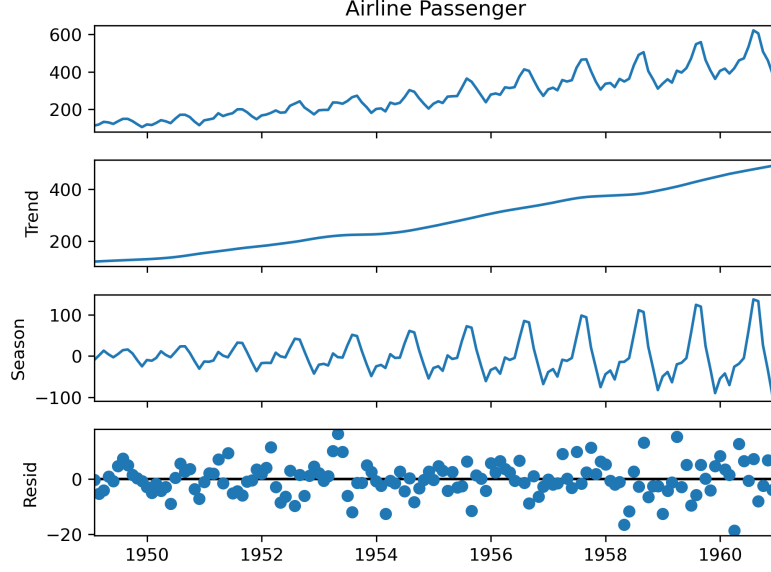
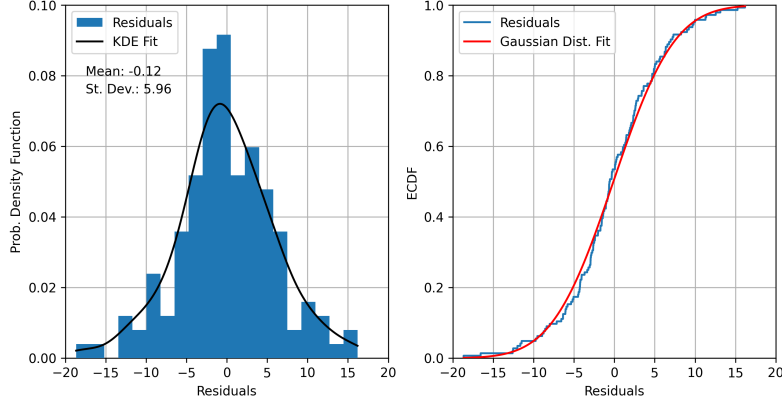


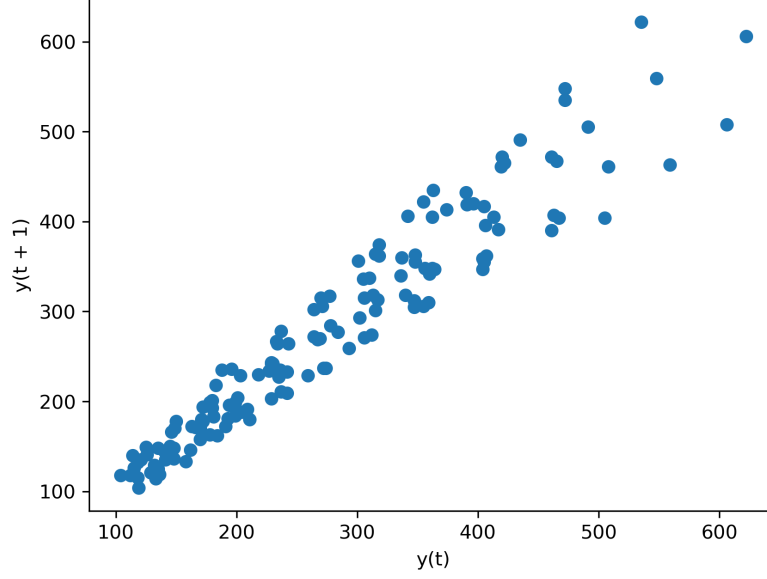
Fig. 2 Distribution of the Time-Series Residual



of $\mathbb{E}[Y]$, contrary to a full modeling of stochastic process such as Polynomial Chaos and Kosambi–Karhunen–Loève theorem. The likelihood function $L(g(x)|x)$ is defined as a function that measures how likely (i.e., close) is the model prediction $g(x)$ given x is close to the observations. That is, the likelihood is maximized at the expected value of the distribution of the observation at a certain t . The selection of $L(g(x)|x)$ depends on the residual (i.e., randomness) of the observations and the correlation structure between each time step.

The strategy employed in this study comprises of defining the LSTM model architecture, followed by determining the prior and likelihood function that best describes

Fig. 3 Correlation between current value and previous value



the data population, next, the LSTM model is then trained (i.e., backpropagation) on the training data to yield the best-point estimate and then use these values as the starting chain and employ the MTM solver to retrieve the posterior. The total number of parameters used in this study is 4,513 and the lookback value is 5.

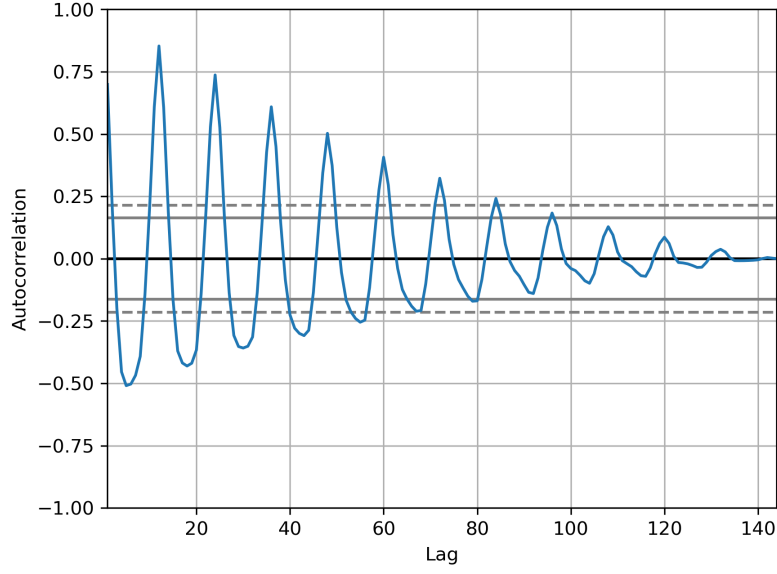
Similarly, a lag value of 5 (i.e., using the previous 5 consecutive value to predict the current value) was selected to mimic the LSTM lookback value. On the other hand, a period of 12 and length parameter of 3 was used in the GPR periodic kernel. Naturally, while the MTM sampler can be also applied on the AR(5) model, this study treated the AR(5) model as the baseline and thus, only deterministic approach was used in determining model parameters ϕ_i .

The discussion on prior distribution used for the parameters is often can be subjective and challenging to determine. Under a different scenarios, a bootstrap sampling technique on the training data (i.e., sample n out of N where $n < N$) can be employed and model training on these sets of bootstrapped samples can be done to retrieve the distributions on each parameters. However, such method cannot be employed here since LSTM depends heavily on the structure of the sequence and consequently, produce a very strong and biased prior. Therefore, a uniform prior of all of the model parameters is used in this study.

3 Result

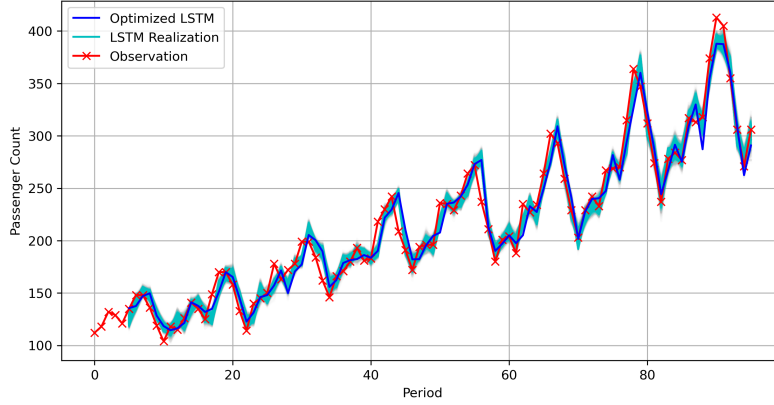
To accelerate convergence, the starting point of the Markov Chain is retrieved from the parameters value retrieved from optimizing the LSTM model through back-propagation (i.e., LSTM parameterized with the optimized values yields the maximum likelihood with respect to the data). A total of 50,000 samples were collected from

Fig. 4 Autocorrelation of seasonality and residual components of the timeseries data



the MTM sampler and the convergence of Markov chain is determined by a "burn-in" point where the chain reaches a stationary condition. Once the posterior distribution is established, Monte Carlo simulation, sampling from the posterior and feeding it into the LSTM forward model was conducted to retrieve the ensemble of realization of $\mathbb{E}[Y]$. Fig 5 shows the ensemble over the training data along with the optimized LSTM model. The nonparametric model GPR was able to capture most of the obser-

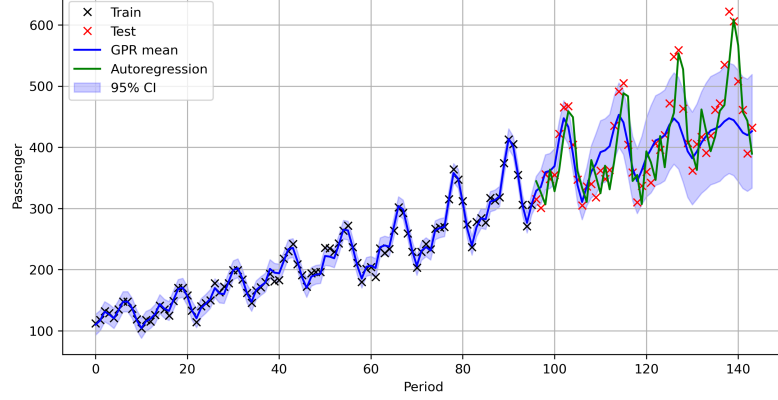
Fig. 5 Ensemble of Realization over the Training Data



vation points within its 95% confidence interval and its prediction within the initial. However, its inability to capture the increasing amplitude of the seasonality might

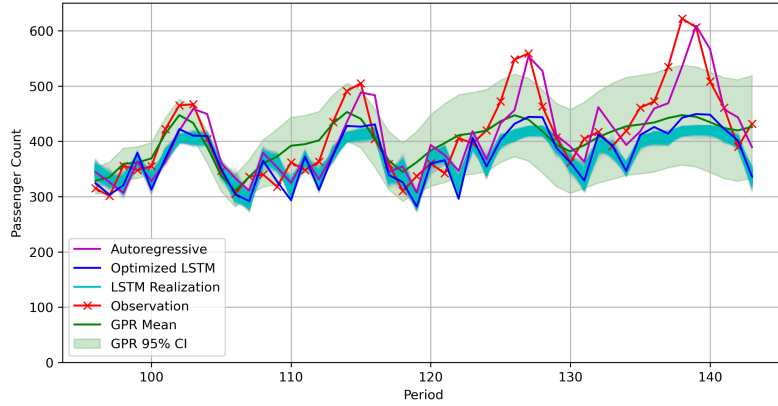
suggest that an improvement in kernel design is necessary or an update-based training might be deployed to improve the model’s long-horizon prediction. On the other hand, the AR(5) model perform exceptionally well in capturing the overall trend and the randomness in the dataset. Fig 6 summarizes the findings above.

Fig. 6 GPR prediction and Autoregression result on both training and testing dataset



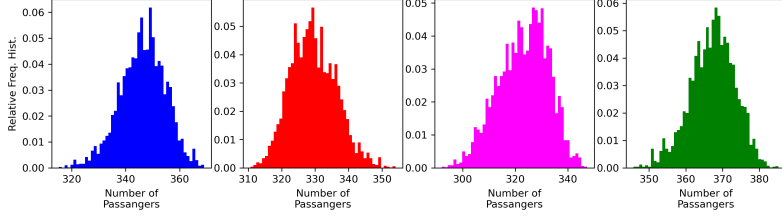
Furthermore, to illustrate and compare the "forecasting" capability (the term "forecasting" is oftentimes misused to replace extrapolation; the LSTM model, given the premise of this paper, can simply extrapolate into the future, for more clarity on "forecasting", refer to Geweke and Whiteman [2]), ensemble over the testing data is also simulated and is presented in Fig 7 below along with GPR and AR(5) prediction.

Fig. 7 Ensemble of LSTM Realization, GPR, and AR(5) on the Testing Data



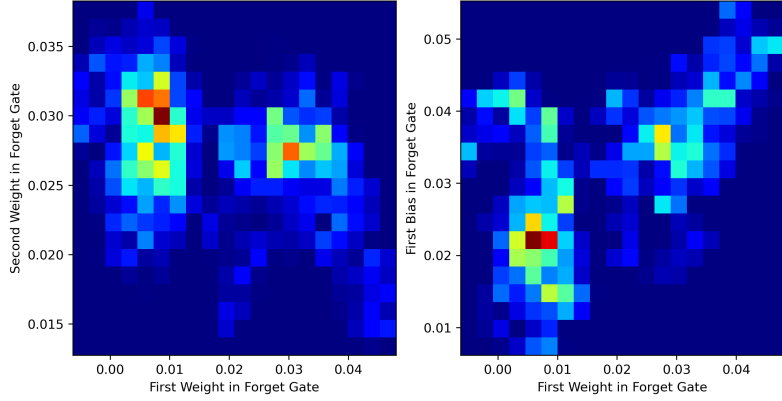
The relative frequency distribution of the LSTM prediction over the first 5 time steps (i.e., $t + 1, \dots, t + 5$) are also presented below in Fig 8

Fig. 8 Relative Frequency Histogram of the first few time steps



One of the benefits of Bayesian inference is the ability to learn the model’s parameters behavior and influence over each other. From the posterior, a joint histogram is created to see if there is any recognizable pattern. As such, it is impractical to show the entirety of the every possible joint histogram between the 4,513 parameters. To illustrate its usefulness, the joint histogram of the first and the second weight parameter in forget gate and the joint histogram of the first weight parameter and the first bias parameter in forget gate are shown below in Fig 9. This further illustrates the strength of Bayesian paradigm, as studying the joint histogram on how each parameter interacts enables a deeper understanding on the model behavior and sensitivity.

Fig. 9 Joint Histogram: A bimodal distribution is observed in the joint distribution



4 Conclusion

One of the main challenges in Bayesian neural network is the sheer number of parameters that need to be calibrated. Given that the parameter space is of multi-dimensional and is large, convergence is oftentimes not easily achieved. Given more time, more simulations can be conducted in order to get better representative sample of the posterior distribution. Fine-tuning the LSTM model to achieve the best balance of network size and predictive error must also be done in order to accelerate convergence and improve model performance.

As such, the joint histogram can provide powerful and deep insight into model’s behavior and sensitivity on each parameter. Furthermore, if relationship between each parameters can be quantified, it offers another insights into optimizing model topology.

References

- [1] Jebb, A.T., Tay, L., Wang, W., Huang, Q.: Time series analysis for psychological research: examining and forecasting change. *Frontiers in Psychology* **6**, 313–320 (2015)
- [2] Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Computation* **9**(8), 1735–1780 (1997)
- [3] Zheng, J., Huang, M.: Traffic flow forecast through time series analysis based on deep learning. *IEEE Access* **8**, 82562–82570 (2020)
- [4] Jospin, L.V., Laga, H., Boussaid, F., Buntine, W., Bennamoun, M.: Hands-on bayesian neural networks—a tutorial for deep learning users. *IEEE Computational Intelligence Magazine* **17**(2), 29–48 (2022)
- [5] Chandra, R., He, Y.: Bayesian neural networks for stock price forecasting before and during covid-19 pandemic. *PLoS ONE* **16**(7), 0253217 (2021)
- [6] Liu, J.S., Liang, F., Wong, W.H.: The multiple-try method and local optimization in metropolis sampling. *Journal of the American Statistical Association* **95**(449), 121–134 (2000)